WIRTSCHAFTS UNIVERSITÄT WIEN VIENNA UNIVERSITY OF ECONOMICS AND BUSINESS

EFMD EQUIS ACCREDITED

Bachelor Thesis

Comparing Machine Learning Approaches for Trade Intent Classification from Noisy Livestream Transcripts

Andreas Oberdörfer

Date of Birth: 24.12.2002
Student ID: h12217505

Subject Area: Information Business

Studienkennzahl: J 033 561

Supervisor: Johann Mitlöhner

Date of Submission: 12.06.2026

Department of Information Systems & Operations Management, Vienna University of Economics and Business, Welthandelsplatz 1, 1020 Vienna, Austria

DEPARTMENT FÜR WIRTSCHAFTSINFORMATIK UND OPERATIONS MANAGEMENT DEPARTMENT OF INFORMATION SYSTEMS AND OPERATIONS MANAGEMENT

Contents

List of Figures

List of Tables

Abstract

Day traders often stream their work live on platforms like YouTube, talking through their trades as they happen. Hidden inside this constant talk are the moments that matter: when they buy, when they sell, and when they move a stop. Pulling out these signals automatically means first turning the speech into text with automatic speech recognition (ASR), and then sorting each line of text into a trading action. This is hard, because the speech contains recognition mistakes, casual everyday language, and far more chatter than actual trade calls.

This thesis compares five machine learning methods on the same task: sorting lines from real livestream transcripts into one of seven trading actions. Three are classical methods (logistic regression, a support vector machine, and a small neural network) that work on simple word-count features called TF-IDF. The other two are modern transformer models (DistilBERT and ModernBERT), used here in a lightweight way: their inner workings are kept fixed, and only a small classifier is trained on top. All five are tested on the same curated benchmark of labelled transcript lines with the same procedure, so the comparison between the models is fair.

The simple word-count methods win. Logistic regression performs best overall, reaching a macro-averaged F1 score of 0,70, and it tells action lines from non-action lines with an F1 of 0,89. A closer look shows why: a handful of trading keywords does most of the work, and the fixed transformer models do not capture this specialised vocabulary any better. The two transformers score lower, even though they are far larger models, because their fixed inner layers were never adjusted to this task. Action by action, clear keyword-based actions such as trimming a position, exiting, or moving a stop are easy to detect, while entries (going long or short) stay hard, because traders phrase them in so many different ways. The main lesson is that on a small, specialised dataset with repetitive phrasing, simple word-count features stay highly competitive, and the extra complexity of a transformer is not worth it unless the model is actually trained on the task. A closing test on five complete, held-out sessions shows the limit of these numbers: asked to read every line of a real stream, even the best model produces far more false alarms than real signals, so by itself it is not deployable. The production system therefore uses the model only in a supporting role: a set of strict, hand-written text rules makes the trade decisions, and the model confirms or vetoes them. Replaying that combined system over the same sessions shows the layered design working: it fires about six times per hour instead of 225, and about one fire in six points at a real trade action.

Keywords: text classification, trade intent detection, TF-IDF, support

vector machine, logistic regression, transformer models, DistilBERT, ModernBERT, automatic speech recognition, livestream transcription

1 Introduction

1.1 Motivation

The growth of live-streaming has created a new kind of real-time financial commentary. On YouTube and Twitch, day traders share their screens and talk through their trades as they happen, calling out when they buy, when they sell, when they adjust a position, and when they move a stop-loss, all in natural spoken language. For a viewer who wants to follow or copy these trades, the problem is easy to state: the useful signals are buried in hours of talking that also include market analysis, replies to chat, personal stories, and long silences. Spotting the exact moment a trader commits to a trade takes constant attention, and the moment a viewer looks away, a signal can slip by. In day trading, timing often decides whether a trade ends as a win or a loss, so a signal noticed late is worth little.

These streams are popular. During market hours, well-known day-trading channels draw thousands of viewers at once, many of them following a single instrument such as the E-mini Nasdaq futures (NQ). But unlike written trade alerts posted on social media, what a streamer says is short-lived and unstructured. The line between just thinking out loud ("I'm watching this level") and actually committing ("I'm long here") can be subtle and depends on the context.

This is what motivates an automated system that can do three things: (1) turn the live audio into text as it happens, (2) decide which lines of that text actually contain a trade signal, and (3) sort each signal into a concrete action, such as "enter long," "trim position," or "move stop." The first step, turning speech into text, has been transformed by recent large models such as Whisper [26], which turn speech into text with an accuracy close to a human transcriber. The second and third steps are a text classification problem: given a short line of transcript and the current trading context, choose one label from a fixed list.

At its heart, this is text classification, one of the most studied problems in natural language processing (NLP) [14]. Classical methods that count words and feed them to a simple linear model have long done well on tasks like sorting positive reviews from negative ones, or spam from real email [11]. More recently, large pretrained models called transformers, such as BERT [6], have set new records on many NLP benchmarks by learning rich, context-aware representations of words from huge amounts of text. But bigger is not always better. On small, specialised datasets this advantage can vanish, because a model trained on general text may simply miss the niche vocabulary that actually matters here [14]. That raises the central question: is the extra

complexity of a transformer worth it for a task like this one?

Several problems show up at once in this domain. First, the language is casual and full of trader slang ("peel some off," "add to the runner," "move to breakeven"). Second, the speech-to-text step adds its own mistakes: specialist terms and sound-alike words are often misheard, in ways that change how the text looks to a classifier; Section 2.7 shows concrete examples. Third, the classes are very unbalanced, because most of what a trader says is commentary, not an actual trade call. Fourth, the same action can be said in many ways: a trader going long might say "I'm long here," "let's try it," "small size," or just "here we go." Finally, the dataset built for this work is small by modern NLP standards, with fewer than 1.500 labelled examples. Taken together, these traits make the task a well-suited test case for comparing classical and modern model families under realistic conditions.

1.2 Research Question

The central question addressed in this work is:

> *Which approach gives the most accurate trade-intent classification on noisy livestream transcripts: classical TF-IDF methods or pretrained transformer models? And on a small, specialised dataset, does the extra complexity of a transformer actually buy any measurable improvement over the simpler methods?*

To answer this, five models, three classical and two transformer-based, are compared under identical conditions: trained and tested on the same labelled data and scored with the same procedure, so that any difference in results comes from the model itself and not from the setup. Two follow-up questions are also examined: (1) Which trading actions are easiest to detect automatically, which are hardest, and why? (2) When a model gets a line wrong, what kinds of mistakes does it tend to make, and how do those mistakes relate to the way traders speak?

1.3 Scope

The scope of this thesis is deliberately narrow. The classification task is studied on its own: each model receives a short, ready-made piece of text containing one transcript line plus its trading context, and returns a single predicted action. The speech-recognition step before it and the trade-execution step after it are described for context but are not tested or tuned here. Only English-language streams are used. All labelled data comes from recorded

streams of a single YouTube streamer, FlowzoneTrader, who day-trades the E-mini Nasdaq futures (NQ), so the results may not carry over to other traders, instruments, or styles.

All transformer models are also used with their encoder frozen: the large pretrained part of the model that reads the text stays unchanged, and only a small classifier on top is trained on the labelled data. Fully retraining the transformers (called fine-tuning) is not explored here, because it would need a much larger dataset to work safely. The same applies to LoRA, a lighter-weight form of fine-tuning that adjusts only a few small add-on weights inside the model instead of all of them; it is discussed as future work (Section 5.7).

1.4 Structure of This Work

The remainder of this thesis is organized as follows. Section 2 introduces the concepts this work builds on, from text classification with TF-IDF features to transformer models and automatic speech recognition, and reviews related research on NLP in financial markets. Section 3 describes the experiment: the system around the classifier, the exact classification task and its labels, how the dataset was built, checked, and cleaned, the five models, and the evaluation procedure. Section 4 presents the results, Section 5 discusses what they mean and where their limits lie, and Section 6 summarizes the question, the procedure, and the findings.

2 Related Work

This section reviews the methods and research areas that form the foundation of the present work. The discussion proceeds from general text classification concepts to the specific model families evaluated, then covers the speech recognition technology that generates the input data, and concludes with a survey of NLP applications in financial markets.

2.1 Text Classification

Text classification, the task of assigning predefined labels to natural-language documents, is a foundational NLP task spanning spam filtering, sentiment analysis, topic categorization, and intent detection [14]. Approaches differ mainly in how text is represented (engineered features versus learned representations) and in the complexity of the classifier (linear versus nonlinear).

Early opinion-mining work by Hu and Liu [10] showed that sentiment could be extracted from unstructured reviews with relatively simple pattern-based

and statistical methods, establishing that domain-specific lexical patterns are often highly predictive even without deep syntactic or semantic analysis; later surveys such as Percha's clinical-text-mining review [22] trace the evolution from rule-based systems to neural methods while stressing that the choice should be guided by the data rather than the novelty of the method. A consistent finding is that the quality and representativeness of the training data often matter more than the algorithm [14]: small, noisy, or imbalanced datasets can nullify the advantages of sophisticated models, while careful curation and feature engineering can compensate for architectural simplicity. This foreshadows the central tension of the present work: whether the richer representations of pretrained transformers overcome the disadvantage of a small, noisy, domain-specific corpus, or whether the corpus characteristics dominate.

2.2 Feature Extraction: TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) is a classical text representation scheme that assigns a numerical weight to each word in a document based on two factors: how frequently the term appears in the document (term frequency) and how rare it is across the entire corpus (inverse document frequency) [17, 28]. Formally, for a term t in a document d within a corpus D:

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \times \log\left(\frac{|D|}{|\{d' \in D : t \in d'\}|}\right) \tag{1}$$

where $\text{tf}(t, d)$ denotes the frequency of term t in document d, and the logarithmic factor downweights terms that appear in many documents. Sublinear scaling of the term frequency component ($1 + \log(\text{tf})$ instead of raw tf) is commonly applied, so that a term occurring ten times counts only a little more than one occurring twice [17].

Because the word *feature* appears throughout this thesis, it is worth pinning down early. A feature is one measurable property of the input that a model can use for its decision. In TF-IDF, every distinct word seen in the training texts becomes one feature, and its value for a given document is the weighted count from Equation 1. Pairs of neighbouring words are usually included as features as well: a single word is called a *unigram*, a two-word sequence a *bigram*. The line "taking a partial here" thus switches on the unigram features "taking," "partial," and "here" and the bigram features "taking partial" and "partial here" (very short words such as "a" are skipped). Each document becomes a long list of numbers, one per feature,

almost all of them zero. Despite its simplicity, this representation has proven remarkably effective for text classification, particularly when combined with linear classifiers [11]: it is interpretable (each feature is a human-readable word or phrase), cheap to compute, and it picks up domain-specific vocabulary directly from the training data, with no need for pretrained models or word lists from outside.

TF-IDF has one big weakness: it treats every word on its own. It ignores word order, and it ignores meaning, so two lines that mean the same thing but use different words share no features at all. For example, "I'm long here" and "just bought a few" both signal a buy, but because they have no words in common, TF-IDF sees them as completely unrelated. This is exactly the gap that word embeddings, covered in the next sections, are meant to close. That said, keying on exact words turns out to suit trading talk very well, because traders tend to repeat the same handful of action words ("trim," "stop," "long"); the feature-importance analysis in Section 4.6 shows this directly.

2.3 Classical Classification Algorithms

Three classical classification algorithms are evaluated in this work, all applied to TF-IDF feature vectors.

Logistic Regression is a linear model that predicts class probabilities via the softmax function applied to a linear combination of input features [9]. For a feature vector $\mathbf{x}$ and K classes, the probability of class k is:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)} \tag{2}$$

The weights $\mathbf{w}_k$ and b_k are the model's *parameters*: the adjustable numbers inside a model that training tunes. Whenever this thesis says a model has 66 million parameters, it means 66 million such numbers. Training searches for parameter values that minimize the *cross-entropy loss*, a score that measures how wrong the predicted probabilities are: it is small when the model assigns a high probability to the correct class and grows sharply when the model is confidently wrong. Typically a penalty called L2 regularization is added, and since regularization returns several times in this work, a plain-language explanation is useful: the penalty grows with the size of the parameters, which discourages the model from putting extreme weight on any single feature. This is a standard defence against *overfitting*, the failure mode in which a model memorizes its training examples instead of learning patterns that carry over to new data. On small datasets like the one used here, this risk

is high. Logistic regression is a natural baseline for text classification due to its interpretability, native probability output, and strong performance on high-dimensional sparse features [11].

Support Vector Machines (SVMs) find the hyperplane that maximizes the margin, that is, the distance between the closest training examples of different classes, in the feature space [4]. For linearly separable data, the decision function is:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \tag{3}$$

where $\mathbf{w}$ and b are chosen to maximize $\frac{2}{\|\mathbf{w}\|}$ subject to correct classification of all training examples. Real data can rarely be separated perfectly, so in practice a softened version is used: some training examples are allowed to sit on the wrong side of the boundary, and a setting controls how strongly such examples are punished [4]. SVMs have a long history of success in text classification [11]. One practical difference from logistic regression is that an SVM does not output probabilities, only a raw score for each class. When probabilities are needed, a small extra step called Platt scaling is trained afterwards; it converts those raw scores into probabilities between 0 and 1 [25].

Multi-Layer Perceptrons (MLPs) are feedforward neural networks consisting of an input layer, one or more hidden layers with nonlinear activation functions, and an output layer [7]. An MLP with a single hidden layer computes:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \tag{4}$$

where σ denotes a nonlinear activation function (for example ReLU, which simply replaces negative values with zero). Unlike logistic regression and linear SVMs, MLPs can learn nonlinear decision boundaries, which may be beneficial when the relationship between features and labels is not linearly separable. However, additional capacity also increases the risk of overfitting, particularly on small datasets.

2.4 Word Embeddings and Pretrained Representations

Word embeddings take the next step beyond counting words: each word is turned into a list of numbers, called a vector, chosen so that words with similar meanings get similar vectors. Such vectors capture meaning rather than spelling, and even simple arithmetic on them can expose relationships. In the most famous example, taking the vector for “king,” subtracting the vector for “man,” and adding the vector for “woman” lands close to the vector for “queen” [18]. Word2Vec [18] learns such vectors by training a small

network to predict a word from its neighbours; GloVe [21] reaches a similar result from global word co-occurrence counts. Both share one limitation: every word gets a single fixed vector, no matter which sentence it appears in (these are called *static* embeddings). That breaks down for words that carry several meanings. The word "stop," for instance, names a protective sell order in "move my stop" but is an ordinary verb in "stop talking"; a static embedding has no choice but to squeeze both meanings into the same vector. Contextual embeddings remove this limitation by letting a word's vector depend on the sentence around it; ELMo [23] did so with bidirectional language models and improved results on tasks such as question answering and named-entity recognition, leading directly to the transformer models discussed next.

The progression from TF-IDF through static to contextual embeddings is a steady climb toward richer representations of meaning, but each step up costs more in model size and training data.

2.5 Handling Class Imbalance

Class imbalance means some categories have far more training examples than others, and it is a common problem in real-world classification [3]. When the data is very uneven, a classifier can look deceptively good by simply favouring the biggest category: always predicting the most common answer keeps the measured error low, while the rare categories, which are often the ones that matter, are never caught at all.

Several fixes exist. *Oversampling* methods such as SMOTE [3] invent extra synthetic examples of the rare classes. *Undersampling* does the opposite, dropping some examples of the common class to even things out. *Cost-sensitive learning* instead changes the scoring so that mistakes on rare classes are punished more heavily. The `class_weight=balanced` setting used here for logistic regression and the SVM is one example: it gives each class a weight that grows as the class gets rarer, in inverse proportion to how often the class appears. Concretely, the weight given to a class k is:

$$w_k = \frac{n}{K \cdot n_k} \tag{5}$$

Here n is the total number of training examples, K is the number of classes, and n_k is the number of examples in class k itself. So a rare class, with few examples, gets a large weight, and each of its examples counts for more.

This thesis uses a mix of these ideas. When the training data for each experiment is assembled, the dominant `NO_ACTION` class is capped at 2,5 times the total number of action examples (Section 3.8). On top of that, the linear

models and the transformer heads use cost-sensitive class weighting, while the MLP, whose scikit-learn implementation cannot weight classes directly, is balanced by oversampling instead: each rare-class example is simply copied several times in the training set, in proportion to the same class weights, so the network sees the rare classes more often (details in Section 3.7).

2.6 Transformer Models

Before transformers, the dominant neural models for text were *recurrent* networks, which read a sentence strictly one word at a time, carrying a running summary along from word to word. That design has two drawbacks: the words cannot be processed in parallel, which makes training slow, and information from the start of a long passage tends to fade by the time the end is reached. The Transformer architecture, introduced by Vaswani et al. [33], removed the word-by-word reading entirely and replaced it with a *self-attention* mechanism that lets every token (a word or word piece) look directly at every other token at once. This allows parallel computation and makes long-range connections in the text just as easy to use as neighbouring words. The core self-attention operation computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6}$$

where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are query, key, and value matrices derived from the input, and d_k is the dimensionality of the key vectors. Multi-head attention extends this mechanism by running h parallel attention operations with different learned projections, concatenating their outputs, and projecting back to the model dimension. This allows the model to attend to different types of relationships simultaneously.

A complete transformer layer combines this attention step with a small feed-forward network and standard stabilizing components (residual connections and layer normalization). Stacking many such layers produces increasingly abstract representations of the input. The whole stack is called the *encoder*: the part of the network whose job is to read text in and give back, for every token, a vector of numbers that captures what that token means in its particular context. Whenever this thesis speaks of "embedding" a piece of text, an encoder of this kind is doing the work.

BERT (Bidirectional Encoder Representations from Transformers) [6] used the transformer encoder to pretrain a language model on a huge amount of text. Its main training trick is fill-in-the-blank: hide a random word in a sentence and have the model guess it from the words on both sides (this is called masked language modelling). Because it sees the words before and

after the blank at the same time, BERT builds a richer sense of each word than older models that read in only one direction. Once pretrained, BERT can be adapted to a specific task by adding a small output layer and training it on labelled data, a step called fine-tuning. This approach broke records across many standard NLP tests.

There are two ways to put BERT to work on a new task. The first is *fine-tuning*: start from the pretrained weights and continue training the whole model, encoder included, on the labelled task data, so that every layer adapts to the new task. The catch is that retraining all of those weights needs a lot of labelled examples and careful tuning to stop the model from memorising the training set; with a small dataset, that risk is high. The second way is *feature extraction*: freeze the pretrained encoder, use it only to turn text into vectors, and train just a small classifier on top. This gives up the ability to tailor the encoder to the task, but it trains far fewer parameters, which suits a small dataset. The trade-off between these two options has been studied directly [24]. This thesis uses the feature-extraction approach.

DistilBERT [29] is a smaller, faster version of BERT, built with a technique called knowledge distillation [8]: a compact "student" model is trained to copy the behaviour of the larger "teacher" model (here, BERT). The idea of compressing a large model into a smaller one in this way goes back to earlier model-compression work [2]. Crucially, the student learns from the teacher's full set of output probabilities, not just its final answer, so it also picks up which classes the teacher considers similar. The result is about 40% smaller and 60% faster while keeping roughly 97% of BERT's language ability (66 million parameters instead of 110 million).

ModernBERT [35] is a recent redesign of BERT that includes many improvements developed since the original appeared. It can read much longer inputs at once (up to about 8.000 tokens instead of a few hundred), it uses newer and more efficient ways of paying attention to context, and it was trained on far more data, about 2 trillion tokens of text and code, an order of magnitude more than BERT. It is larger than the original BERT (149 million parameters) yet still runs faster, thanks to these efficiency changes, and it sets new top scores among models of its type. The technical names behind these gains (such as rotary position embeddings [31] and faster attention [5]) are not essential here; what matters for this thesis is that ModernBERT is a stronger, more modern encoder than DistilBERT.

These models shine on big standard benchmarks, but how well they do on small, specialised datasets, especially when full fine-tuning is not an option, is still an open question. This thesis adds evidence by testing frozen transformer encoders as feature extractors on a new classification task.

2.7 Automatic Speech Recognition

The input data for the classification task in this work is generated by automatic speech recognition (ASR). The field of ASR has undergone a paradigm shift in recent years, moving from pipeline-based systems (acoustic model, language model, decoder) to end-to-end neural models that directly map audio waveforms to text.

Whisper [26] is a large speech-to-text model from OpenAI, trained on more than 680.000 hours of audio collected from the internet. It works in two stages: one part listens to the audio and turns it into an internal numerical form, and a second part reads that form and writes out the text one word at a time. The training data is "weakly supervised," meaning the matching transcripts were scraped from the web (for example, from video subtitles) rather than carefully typed up by people. Those transcripts contain some errors, but the sheer amount of data outweighs them: the model still learns to transcribe correctly, because the errors are rare and inconsistent while the correct patterns repeat millions of times.

Whisper generalises unusually well to audio it was not specifically trained for (different accents, recording quality, and subject matter), often matching systems tuned for one setting, a property known as *zero-shot* robustness, and on several benchmarks it approaches human transcribers [26]. It is released in several sizes, from a 39-million-parameter model to a 1,5-billion-parameter one, trading speed for accuracy.

Still, Whisper is not perfect, and like any speech-to-text system it slips up on specialist jargon, fast or informal speech, people talking over each other, and poor audio. The kinds of mistakes that matter here are easiest to see through an example. Imagine a trader saying, quickly and mid-thought:

> "okay, VWAP's right here, I'm gonna peel some off and move to breakeven."

A clean transcript would keep every word. In practice, several things can go wrong with this one line. The technical term "VWAP" (a common price indicator) might come out as "the WAP" or "v-wap," because the model has rarely heard it. A normal-sounding word can be replaced by its sound-alike: "peel" (meaning to sell part of a position) might become "peal." Because the trader is rushing, filler words and half-finished phrases get merged or dropped, so "I'm gonna peel some off" may shrink to something shorter and vaguer. And Whisper chops the audio into chunks, so a single sentence can be cut in two, leaving "I'm" at the end of one line and "long here" at the start of the next, which makes that line impossible to classify on its own. The full system stitches neighbouring chunks back together to reduce this.

These errors flow straight into the classification step and act as noise that every model has to deal with. The key point for this thesis is that the text the classifier sees is not just casual and messy; it has also been changed in predictable ways by the speech-to-text step itself. So any accuracy number has to be read with that upstream noise in mind. To reduce the most common errors, a clean-up step is applied between transcription and classification, described in Section 3.3.

2.8 NLP Applications in Financial Markets

The application of natural language processing to financial data has a substantial and growing research literature. Early work by Loughran and McDonald [16] demonstrated that general-purpose sentiment dictionaries perform poorly on financial texts, as words with negative connotations in everyday language (e.g., "liability") carry neutral or positive meaning in financial contexts. This finding motivated the development of domain-specific lexicons and, subsequently, domain-specific language models.

Bollen et al. [1] showed that collective mood states derived from Twitter (today's X) could predict movements in the Dow Jones Industrial Average, establishing a link between textual sentiment and market behavior. More recent work has applied transformer-based models directly to financial texts: FinBERT [38] is a BERT model further pretrained on financial communications that achieves strong performance on financial sentiment classification tasks. Surveys by Xing et al. [37] and Wankhade et al. [34] give an overview of the many NLP applications in finance and sentiment analysis, ranging from news-based prediction to earnings-call analysis.

The present work differs from this prior literature in an important respect: rather than analyzing written financial texts (news articles, regulatory filings, social media posts), it targets *spoken* trading commentary transcribed in real time. The input is not a polished written document but a noisy, fragmentary transcript of rather spontaneous speech, introducing challenges that are more characteristic of dialogue systems and spoken language understanding than of traditional financial NLP.

3 Methodology

This section describes the experiment: the system around the classifier, the exact task and its labels, how the dataset was built and cleaned, how the models are configured, and how they are evaluated.

3.1 System Pipeline Overview

The classification task studied in this work is one component of a larger system designed to automatically detect and execute trades from live YouTube daytrading streams. The complete pipeline consists of four stages:

1. **Audio capture and voice activity detection.** Audio is captured from a browser tab via a React-based frontend and processed by a voice activity detector (based on the WebRTC standard), which splits the continuous audio stream into speech segments and discards silence and background noise.

2. **Speech-to-text transcription.** Speech segments are transcribed with Whisper [26], run through `faster-whisper`, a reimplementation of Whisper built on CTranslate2, a software library specialised in executing neural networks quickly on ordinary hardware. Each segment is transcribed twice: a fast preview pass shows the user a provisional transcript almost immediately, and a more careful final pass produces the text that the rest of the pipeline uses. Both passes run the same Whisper model; the final pass simply spends more computing time on decoding, weighing more alternative readings of the audio before settling on one. A clean-up step then corrects frequent recognition errors in trading terminology (Section 3.3).

3. **Intent classification.** Each finalized transcript segment is classified into a trading action. In the full production system, this decision is shared by three layers. A rule engine holds several hundred hand-written text patterns, so-called regular expressions, which are search patterns that match specific wordings such as "I'm long here," and proposes an initial label. A locally running machine learning classifier then assigns a probability to each possible action for the same text; these confidence scores are used to veto rule matches that look like false alarms, and to recover actions the rules missed. An optional third layer can send ambiguous entry signals to a cloud-based large language model for confirmation. The machine learning classifier in this stage is the component evaluated in this thesis.

4. **Risk validation and trade execution.** Classified trade signals pass through a risk engine that checks for stale signals, insufficient confidence, and position-size limits before orders are submitted to a NinjaTrader 8 trading platform via a local HTTP bridge.

The present work focuses exclusively on stage 3: given a transcript segment and its trading context, predict the correct action label. The other stages are treated as fixed infrastructure. Figure 1 illustrates the complete pipeline architecture.

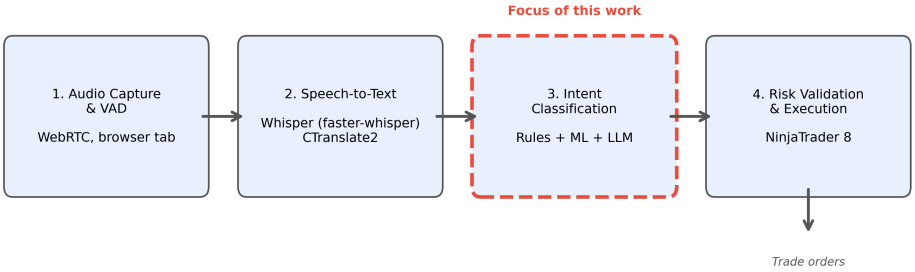

Figure 1: Overview of the four-stage system pipeline. The classification stage (highlighted) is the component evaluated in this thesis. Audio is captured from a browser tab, transcribed by Whisper, classified into trading actions, and validated before execution.

3.2 Task Definition and Label Set

The classification task assigns one of seven labels to each input example. The labels and their definitions are shown in Table 1.

Table 1: Label definitions for the trade-intent classification task.

Label	Role	Description
`NO_ACTION`	Neutral	No trade signal detected
`ENTER_LONG`	Entry	Open a new long position
`ENTER_SHORT`	Entry	Open a new short position
`TRIM`	Management	Reduce position size (partial profit-taking)
`EXIT_ALL`	Management	Close the entire position
`MOVE_STOP`	Management	Adjust the stop-loss level
`MOVE_TO_BREAKEVEN`	Management	Move the stop-loss to the entry price

Each input example is a structured text prompt that encodes the traded instrument, the trader's current position state, and three overlapping windows of transcript text:

```
symbol=MNQ 03-26
```

```
position=LONG
last_side=LONG
recent=there s a nice shove up. we cleared out
analysis=there s a nice shove up. we cleared out there.
         so you got to take a partial
current=there. So, you got to take a partial
```

The three text windows (`recent`, `analysis`, `current`) supply context at different temporal granularities, so the classifier sees not only the current utterance but also what was said just before; the `position` and `last_side` fields encode the current market position, which is essential for telling management actions (which require an open position) from entry actions (which open one). The `symbol` field reads MNQ rather than NQ because the copying system executes its trades on the Micro E-mini Nasdaq contract (MNQ), a smaller-sized version of the NQ contract the streamer trades. This overlapping design addresses a core difficulty of streaming transcription: Whisper cuts audio into chunks of roughly 5–30 seconds, so a single utterance may be cut in half, and the overlap lets the classifier reconstruct the intent even when the decisive words are split across two lines.

Encoding the position state as plain text inside the prompt, rather than as a separate input, is done on purpose: it lets a single text-processing pipeline handle both the words and the trading context. How each model family digests these state lines is described in Section 3.7.

3.3 Text Normalization

Between transcription and classification, a clean-up step corrects the recognition errors that occur most often in this domain, so that the classifier sees consistent text. It consists of 14 hand-written search-and-replace rules plus three general clean-ups, all applied as plain, deterministic text substitutions; no machine learning is involved.

A large share of the rules glues back together terms that Whisper tends to split apart. When the contract names that traders use as shorthand are spelled out letter by letter, “m n q” becomes “mnq” and “e s” becomes “es,” and the price benchmark heard as “v w a p” or even as “view up” becomes “vwap.” A second group of rules fixes sound-alike words in their trading sense, but only when the surrounding words make the trading meaning clear: a trader “paying myself a peace” is corrected to “piece,” and “got my ad on” becomes “got my add on.” Restricting these corrections to specific phrasings keeps the rules from corrupting genuine uses of the original words. A third group standardizes compounds that Whisper writes inconsistently, so “break

even" and "broke even" both become "breakeven," matching the spelling the labelled data uses. Finally, three general clean-ups run over everything: all text is lowercased, typographic apostrophes are replaced with plain ones, and the commas inside numbers are removed, so that a price like "24,600" is read as the single number "24600" rather than as two separate pieces.

Simple as they are, these substitutions reduce the vocabulary fragmentation caused by recognition errors: without them, "vwap," "v w a p," and "view up" would look like three unrelated expressions to the TF-IDF vectorizer and to the transformer tokenizers, even though the trader said the same word each time.

3.4 Dataset Construction

The training dataset was constructed in three stages: AI-assisted labelling, human review, and merging with normalization.

Stage 1: AI labelling. Trading livestream transcripts archived from YouTube were processed by large language models (LLMs), which assigned action labels to each transcript line based on the speaker's words and trading context. Three LLMs were used across different annotation passes: Claude Sonnet produced 278 labelled examples in an initial high-quality pass, Gemini Pro produced 78 examples in a validation run, and Gemini Flash produced 1.139 examples in a bulk labelling pass. This follows an idea from the research literature known as weak supervision: instead of having humans label every line, an automatic but imperfect labeller does the bulk of the work first, and humans then check and correct it [27]. Like any automated annotation, it makes mistakes: an LLM can mislabel sarcasm, hypothetical talk, or a half-finished sentence. The following stages catch many of these errors, but not all of them.

Stage 2: Human review. Each AI-generated label was manually reviewed by the author and either accepted, rejected, or corrected. Review decisions were preserved in the dataset as metadata fields, enabling downstream analysis of labelling reliability. This human-in-the-loop approach combines the scalability of automated labelling with the quality control of manual review, although review reduces errors rather than eliminating them.

Stage 3: Merging and normalization. The three reviewed collections were then combined into a single dataset; in the rare cases where two collections had labelled the same line, the newer label won. During the merge, two older label types were also translated into the final seven-label scheme. Lines where the trader merely talks about a possible future trade ("looking for shorts up here") without acting were relabelled as `NO_ACTION`, since they are commentary rather than an executed action; this affected 437 lines.

And a handful of lines where the trader enlarges an existing position were converted into the matching entry label, because adding contracts is, from an execution standpoint, another entry in the same direction. The resulting dataset contains 1.466 examples drawn from 103 transcripts.

Two caveats apply to all of these labels. First, a label records what the trader *announced*, not what verifiably happened: when a streamer says "I'm long here," the dataset takes that entry at face value. No broker statements or account records were available, so there is no way to prove that an announced trade was actually placed. Second, although every label passed human review, the labels began as AI output, and the dataset cannot be assumed perfectly clean: the AI may have skipped real actions entirely, and since not every transcript was re-read line by line, such missed labels would go unnoticed. Both caveats are taken up again in Section 5.6.

Checking the labels by hand. To see how far the labels can be trusted, they were also spot-checked. The 10 transcripts with the most action labels, around 200 labels in total, were re-read: for every labelled line, the surrounding lines were read to judge whether the label matches what the trader was actually doing at that moment. Almost every checked label held up, and none was clearly wrong. The same check, however, confirmed the missed-label problem from the other direction: the unlabelled lines of those same transcripts contain roughly 230 likely actions that carry no label, most of them repeats of an action announced moments earlier, small profit-takes, or stop adjustments mentioned in passing. In short, what is labelled is almost always right, but not everything that happened is labelled. Some lines labelled `NO_ACTION` may therefore contain weak or repeated actions, which blurs the boundary between action and non-action during both training and evaluation. For this reason, the benchmark in this thesis should be read as a controlled comparison of models on the labelled examples, not as a measurement of how well trade signals are detected in raw livestream speech.

3.5 Dataset Characteristics

Table 2 shows the label distribution of the original dataset.

Table 2: Label distribution in the original dataset (1.466 file entries; 1.407 unique examples after content deduplication).

Label	Count	Percentage
NO_ACTION	569	38,8%
TRIM	267	18,2%
EXIT_ALL	271	18,5%
ENTER_SHORT	153	10,4%
MOVE_STOP	97	6,6%
ENTER_LONG	94	6,4%
MOVE_TO_BREAKEVEN	15	1,0%
Total	**1.466**	**100%**

The distribution is strongly imbalanced, and it is worth being clear about where the NO_ACTION examples come from. They are labelled lines like all the others: partly lines that the AI annotators explicitly marked as non-action commentary, and partly the 437 remapped setup lines described above. They are *not* a random sample of all the chatter in the streams. That is why NO_ACTION makes up only 38,8% of this dataset, even though in a complete, uninterrupted session far more than 38,8% of all spoken lines carry no action; compared to a real stream, the dataset is heavily *action-enriched*. At the other extreme, MOVE_TO_BREAKEVEN has only 15 examples (1,0%), far too few for any classifier to learn reliable patterns. Both points matter for interpreting all results and return in Sections 3.4 and 5.6.

3.6 Data Cleanup

Before model training, an automated cleanup pipeline corrected several recurring flaws in the original 1.466-example dataset.

The largest fix concerns contradictory position states. Some management-label examples (for example TRIM or EXIT_ALL) carried position=FLAT, which is impossible: a trader cannot trim a position that does not exist. These contradictions survived the human review because the review judged whether the action label matched the trader's words; the position fields were filled in automatically during labelling and were not part of that check. In 200 examples, other recorded fields revealed the actual position and the state was repaired; the handful of contradictory examples that could not be repaired was removed.

The cleanup also removed two smaller groups. The first is repeated announcements: livestream speech repeats itself, so the same action sometimes

ended up labelled twice within a few lines ("piece off into 50s" and, two lines later, "partial here, folks" describe one single trim), and only the first occurrence was kept. The second is advice to the audience: a few lines address the viewers rather than describe the trader's own position ("you must pay yourself here"), and labelling those as executed trades would teach the classifier exactly the kind of false positive the system must avoid. After all steps, **1.442 clean examples** from 103 transcripts remain.

One last reduction happens automatically whenever the data is loaded: exact duplicates, entries whose label, prompt text, and position fields are all identical character for character, are collapsed into one. Such doubles arise because overlapping speech chunks can produce the very same prompt twice, and keeping both would let one line count twice in training and testing. This shrinks the original dataset from 1.466 to 1.407 unique examples, and the cleaned dataset from 1.442 to 1.377.

This gives four dataset sizes, which differ only by the two mechanical steps just described. Table 3 lists them side by side.

Table 3: The four dataset sizes used in this thesis. Each row differs from the row above by exactly one step: the cleanup removes 24 flawed examples, and loading the data collapses exact duplicates.

Stage	**Examples**	**Role in this thesis**
Raw merged dataset	1.466	Input to the cleanup pipeline
After cleanup (24 removed)	1.442	Cleaned dataset on disk
Raw, duplicates collapsed	1.407	Baseline (Section 4.2)
Cleaned, duplicates collapsed	1.377	Primary results (Section 4.1)

All example counts and per-label support figures reported in Section 4 refer to the deduplicated totals. The cleaned, deduplicated dataset (1.377 examples) is the primary basis for comparison; results on the original dataset are reported once, to measure the effect of cleanup.

3.7 Model Architectures

This section describes how each model is set up. All models were implemented in Python, using scikit-learn [20] for the classical models and the Hugging Face Transformers library [36] with PyTorch [19] for the transformer models.

3.7.1 TF-IDF Feature Extraction (Shared by Classical Models)

All three classical models share the same text featurization step: TF-IDF vectorization with up to 20.000 features, consisting of unigrams and bigrams (single words and pairs of neighbouring words), with sublinear term-frequency scaling so that very frequent words do not dominate.

The effect is intuitive. A segment like *"taking a partial here on this move up"* contains the bigram "taking partial," which carries a high weight because it appears almost exclusively in `TRIM` examples, giving the classifier a strong signal. By contrast, *"we got a nice shove up, let's try it"* contains no explicit trading keyword; phrases such as "let's try" appear across many classes and carry low weight, which is exactly why vaguely phrased entries are harder to classify than explicitly announced management actions.

The choice of 20.000 maximum features balances vocabulary coverage against dimensionality. With fewer than 1.500 training examples, a much larger vocabulary would mostly add features observed in only one or two examples, inviting overfitting. The inclusion of bigrams is particularly important for this domain: many trading actions are signalled by two-word phrases ("small long," "move stop," "I'm out," "peel off") that would be indistinguishable at the unigram level.

The structured prompt format means that the trading context is encoded by these same features. The vectorizer splits tokens at characters such as "=", so a state line like `position=LONG` contributes the word pair "position long" as a bigram feature, and `position=FLAT` contributes "position flat." Through these features, a model can learn, for example, that management actions go together with an open position; the feature-importance analysis later confirms that exactly these state bigrams carry substantial weight. In effect, the structured prompt turns a two-part problem (text plus trading state) into a single text classification problem that word-count features can handle.

3.7.2 Logistic Regression

Logistic regression is used as the first baseline, in the standard configuration of its scikit-learn implementation: a moderate amount of L2 regularization, so that no single word can dominate the decision, and balanced class weighting, so that the rare classes count more during training. It is the simplest model in the comparison, it outputs probabilities directly, and it handles the class imbalance out of the box.

3.7.3 Support Vector Machine

The linear SVM (`LinearSVC` in scikit-learn) uses the same feature vectors and the same balanced class weighting. Because an SVM produces only raw scores, the Platt-scaling step described earlier is trained on top of it to turn those scores into probabilities. The SVM draws its boundary to keep the widest possible margin between the classes, which can generalize better when training examples are scarce.

3.7.4 Multi-Layer Perceptron

The MLP is a two-layer feedforward neural network (hidden layers of 256 and 128 units) trained with the Adam optimizer [12]. Because scikit-learn's `MLPClassifier` cannot weight classes directly, balance is achieved by over-sampling, and the mechanics are simple: every training example is duplicated in proportion to its balanced class weight. An example of the most common class appears once, while an example of a class that is, say, eight times rarer is copied about eight times, so the network simply sees the rare classes more often. Early stopping (halting training once performance on an internal validation split stops improving) is deliberately left off: the duplicated copies would land on both sides of that internal split, making the validation half look deceptively easy.

3.7.5 DistilBERT (Frozen Encoder + Trained Head)

The DistilBERT model (`distilbert-base-uncased`, 66 million parameters, 6 layers) is used as a frozen feature extractor [29]: the pretrained model itself is never changed and serves purely as a converter from text to numbers. The journey of one example is short, and Figure 2 shows it as a picture. The prompt text is first split into tokens. The frozen encoder then reads all tokens together and produces, for each one, a vector of 768 numbers describing that token in its context. These per-token vectors are averaged into a single vector of 768 numbers, a compact numerical summary of the whole example. Finally, that summary goes into the classification head, the only part that is trained: it rescales the 768 numbers to a stable range and computes one score per label, and the label with the highest score is the prediction.

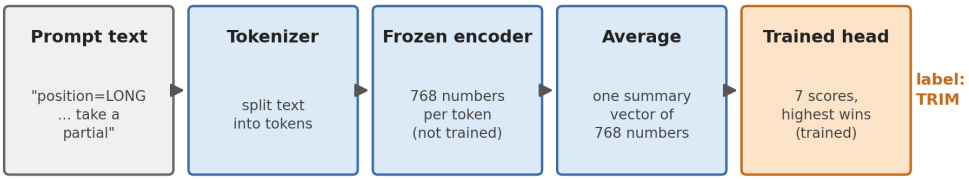

Figure 2: From text to prediction with a frozen encoder. Only the small classification head on the right is trained on the labelled data; the encoder stays exactly as pretrained.

The head is deliberately tiny: it contains only about 6.900 trainable parameters, against the 66 million frozen ones. It is trained like any small neural network, using the cross-entropy loss introduced earlier to measure how wrong the predictions are, with rare classes weighted more heavily so the head cannot ignore them, and an optimizer (AdamW [15]) that adjusts the 6.900 numbers over up to 20 passes through the training data.

Freezing the encoder is motivated by the small dataset. Full fine-tuning would mean adjusting 66 million parameters with only about 1.400 examples, roughly 45.000 adjustable numbers per example, and with that much freedom the model would mostly memorize the training set instead of learning patterns that transfer. Training only the head keeps the trained part at about five parameters per training example, which suits a controlled comparison on a corpus this size.

3.7.6 ModernBERT (Frozen Encoder + Trained Head)

The ModernBERT model (`answerdotai/ModernBERT-base`, 149 million parameters, 22 layers) is used in exactly the same way as DistilBERT: frozen encoder, averaged embeddings, and the identical trained head with the identical training procedure [35]. The only difference is the encoder itself. ModernBERT has a newer design and was pretrained on far more text (about 2 trillion tokens, versus roughly 3,3 billion for the original BERT), so the 768-number summaries it produces should, in principle, capture meaning better. Whether they actually help on this task is precisely what the comparison measures: despite having more than twice DistilBERT's total parameters, ModernBERT has the same number of *trainable* parameters here, since only the head is trained. Any advantage must come from the quality of its frozen embeddings.

3.7.7 Model Comparison Summary

Table 4 summarizes the key properties of all five models.

Table 4: Summary of model architectures and key properties.

Model	**Total parameters**	**Input representation**	**Task-trained parameters**	**Class-imbalance handling**
Logistic regression	∼140.000	TF-IDF, up to 20.000 features	All	Balanced class weights
Linear SVM	∼140.000	TF-IDF, up to 20.000 features	All	Balanced class weights
Multi-layer perceptron	∼5,4 million	TF-IDF, up to 20.000 features	All	Balanced oversampling
DistilBERT frozen head	66 million + 6.900	Token sequence	Head only (6.900)	Weighted loss
ModernBERT frozen head	149 million + 6.900	Token sequence	Head only (6.900)	Weighted loss

One contrast in this table is worth spelling out, because it shapes the results. The classical models train every one of their parameters on the task: logistic regression, for example, can adjust up to 140.000 weights (7 classes × 20.000 features), of which roughly half are active once the actual vocabulary saturates at about 10.000 features (Section 4.11). The transformers, by contrast, keep more than 99,99% of their parameters frozen and adapt only the 6.900 weights of the head. So although the transformers are vastly bigger models overall, the part that can actually specialize in trading language is about an order of magnitude *smaller* than in the classical models. How much this matters is examined in the results and the discussion.

3.8 Evaluation Protocol

3.8.1 Cross-Validation with Transcript-Level Splits

With fewer than 1.500 labelled examples, holding out one fixed test set would be wasteful and fragile: much of the scarce data would be locked away from training, and the result would depend on which lines happen to land in the

held-out set. The standard way around this is *cross-validation* [13]. The data is split into five parts, called folds. The models are then trained five times, each time on four of the parts, and tested on the remaining fifth. Every example is therefore used for testing exactly once, always by a model that never saw it during training, and the five test scores are averaged. All models in this thesis are evaluated this way.

One detail of the split is critical. In standard cross-validation, individual examples are shuffled randomly into folds. That is fine for unrelated examples, but the examples here are not unrelated: lines from the same transcript share the same speaker, the same market day, the same audio quality, and verbal habits that recur throughout a session. If lines from one transcript landed in both the training and the test set, a model could score well partly by recognizing the session rather than understanding the language, and the measured accuracy would be too optimistic. This effect is known as data leakage. To rule it out, entire transcripts are assigned to folds, never single lines, so a model is always tested on sessions it has never seen. The assignment itself is deterministic (it uses a fixed hash of each transcript's file path), so every rerun produces the same folds. The 103 transcripts spread across the 5 folds with roughly 20 transcripts each, though the number of examples per fold varies with how long each session ran and how actively the trader traded in it (exact figures in Section 3.8.3).

Within each fold, the training set is rebalanced by downsampling `NO_ACTION` examples, so that their number does not exceed 2,5 times the total number of action examples. The choice of which `NO_ACTION` examples to keep is made by sorting them by a fixed scrambling code (a hash) of each example's identity and keeping the first ones; this behaves like a random draw but produces exactly the same selection on every rerun. The downsampling applies only to the training part of each fold; the test part keeps the proportions of the labelled dataset. Because the dataset is action-enriched (Section 3.4), these proportions should not be read as the true frequency of action and non-action speech in an uninterrupted livestream.

3.8.2 Metrics

Five metrics are reported, following established practice for imbalanced multi-class classification [30].

Accuracy is the fraction of correctly classified examples. It is simple but can mislead on imbalanced data: a classifier that always predicts `NO_ACTION` would reach about 39% accuracy here without detecting a single trade.

Macro F1 is the unweighted mean of the per-class F1 scores, treating all seven classes equally. A model that ignores the rare classes is punished visibly

by this metric, which makes it a more balanced assessment than accuracy. The F1 score itself is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

Action F1, the primary metric, reduces the problem to one binary question: did the system recognize that *some* trade action occurred? Any action prediction on an action example counts as a true positive; an action prediction on a `NO_ACTION` example is a false positive; a `NO_ACTION` prediction on an action example is a false negative. This directly measures the system's ability to separate actionable signals from commentary, which is its central job.

Action precision is the fraction of all action predictions that were truly actions; high precision means few false alarms. **Action recall** is the fraction of all true actions that were predicted as some action; high recall means few missed signals.

One more term appears throughout the result tables: the *support* of a class is simply the number of test examples whose true label is that class. It says how much evidence stands behind each per-class score; a score computed over 13 examples (as for `MOVE_TO_BREAKEVEN`) is far less stable than one computed over 548.

3.8.3 Benchmark Artifacts

All results in this thesis are read from stored result files written by the benchmark scripts, and the repository contains all of the code needed to regenerate them. The underlying data is a different matter: the labelled dataset and the full transcript archive are too large for the public repository, so only five example transcripts are included to show the data format. An outside reader can therefore inspect every piece of code and every stored result, but rerunning the experiments requires building a dataset of one's own.

Inside the pipeline, two safeguards matter most. The TF-IDF vocabulary is learned inside each training fold only and never from test data, and all splits and seeds are fixed, so reruns produce the same folds (only the transformer heads can vary minimally between reruns, due to random initialization).

Across the five folds, the split assigns 21, 21, 21, 21, and 19 transcripts (225, 339, 287, 293, and 233 test examples, respectively), for 1.377 examples in total. Table 5 lists how often each label occurs among these test examples, summed over all folds; these are the support columns of the per-label tables in Section 4.3.

Table 5: Per-label support: the number of test examples per class on the cleaned dataset, summed across all five folds.

Label	Support
NO_ACTION	548
EXIT_ALL	244
TRIM	242
ENTER_SHORT	144
MOVE_STOP	94
ENTER_LONG	92
MOVE_TO_BREAKEVEN	13

4 Results

This section presents the results in three parts. First come the main scores of all five models on the cleaned dataset, together with a comparison against the uncleaned dataset that shows what the cleanup was worth. The middle subsections then take the results apart to understand them: scores per label, the typical confusions, which words drive the decisions, statistical tests, and an error analysis. Finally, the last two subsections leave the labelled dataset behind and test the system on five complete livestream sessions, first the best model on its own and then the full production pipeline.

4.1 Primary Results on the Cleaned Dataset

Table 6 presents the five-fold cross-validation results on the cleaned dataset (1.377 unique examples after content deduplication, 103 transcripts). Values are reported as mean ± standard deviation across folds. For this primary evaluation, the MLP uses the balanced oversampling procedure described in Section 3.7.

Table 6: Five-fold cross-validation results on the cleaned dataset (primary evaluation). Bold indicates best per column.

Model	Accuracy	Macro F1	Action F1	Action precision	Action recall
Linear SVM	**0,795**±0,021	0,688±0,062	0,884±0,011	**0,900**±0,031	0,870±0,020
Logistic regression	0,785±0,021	**0,703**±0,045	**0,887**±0,024	0,863±0,033	0,914±0,024
MLP	0,763±0,024	0,634±0,049	0,850±0,006	0,898±0,037	0,809±0,031
ModernBERT	0,727±0,012	0,649±0,060	0,872±0,031	0,833±0,044	**0,917**±0,037
DistilBERT	0,676±0,065	0,588±0,029	0,848±0,034	0,801±0,070	0,909±0,071

The two linear classical models achieve the strongest aggregate performance. Logistic regression reaches the best macro F1 (0,703) and action F1 (0,887), while the SVM reaches the best accuracy (0,795) and action precision (0,900). ModernBERT has the highest action recall (0,917), but this comes with lower precision and lower macro F1. The MLP, after balanced oversampling, occupies a middle position: it is more conservative than the transformers and less accurate than the two linear models.

4.2 Original Dataset Baseline and Cleanup Effects

To quantify the impact of data cleanup, every model was also evaluated on the original dataset (1.407 unique examples after content deduplication, 103 transcripts) using the same protocol. These original-dataset scores serve only as a secondary baseline and are not used for the per-label and error analyses; rather than reproduce all five rows here, the effect of cleanup is reported directly as a difference. Table 7 gives, for each model, the change in the three central metrics (cleaned minus original). For the MLP, the unbalanced configuration is used on both datasets, so the cleanup effect is not mixed with the separate balancing change.

Table 7: Cleanup-only performance change (cleaned minus original). Positive values indicate improvement. The MLP row uses the unbalanced MLP configuration on both datasets, so the cleanup effect is not mixed with the later balancing change.

Model	**Δ Accuracy**	**Δ Macro F1**	**Δ Action F1**
Linear SVM	+0,020	+0,017	+0,014
Logistic regression	+0,022	+0,024	+0,016
ModernBERT	+0,048	+0,047	+0,017
DistilBERT	+0,038	−0,006	+0,011
MLP (unbalanced)	−0,006	−0,019	−0,005

Cleanup improved most model metrics. ModernBERT gained +0,048 accuracy and +0,047 macro F1, compared to +0,022 and +0,024 for logistic regression. DistilBERT improved in accuracy and action F1 but decreased slightly in macro F1. The unbalanced MLP did not benefit from cleanup alone, which indicates that its earlier weakness was driven more by class-imbalance handling than by the specific noisy examples removed during cleanup. The separate effect of balanced oversampling on the MLP (Section 4.8) is sizeable: on the same cleaned dataset it raises accuracy from 0,726 to 0,763, macro F1 from 0,572 to 0,634, and action F1 from 0,804 to 0,850. The gains from cleanup are modest in absolute terms, but they make the classic point of garbage in, garbage out: the same models, given slightly cleaner data, learn measurably better.

4.3 Per-Label Performance

Table 8 shows the per-label precision, recall, and F1 for the two linear models on the cleaned dataset, aggregated across all five folds.

Table 8: Per-label precision (P), recall (R), and F1 for the two TF-IDF linear models on the cleaned dataset. Supp. = support, the number of test examples whose true label is the class in question.

	Logistic regression			Linear SVM			
Label	**P**	**R**	**F1**	**P**	**R**	**F1**	**Supp.**
NO_ACTION	0,857	0,777	0,815	0,812	0,850	0,831	548
ENTER_LONG	0,574	0,717	0,638	0,684	0,587	0,632	92
ENTER_SHORT	0,614	0,618	0,616	0,676	0,521	0,588	144
TRIM	0,786	0,835	0,810	0,797	0,860	0,827	242
EXIT_ALL	0,811	0,861	0,835	0,829	0,857	0,843	244
MOVE_STOP	0,818	0,862	0,839	0,833	0,851	0,842	94
MOVE_TO_BREAKEVEN	0,800	0,308	0,444	0,750	0,231	0,353	13

A clear hierarchy of difficulty emerges. Management labels (TRIM, EXIT_ALL, MOVE_STOP) achieve F1 scores between 0,810 and 0,839, reflecting the use of explicit, distinctive keywords ("partial," "taking profit," "I'm out," "move my stop"). Entry labels (ENTER_LONG, ENTER_SHORT) are considerably harder, with F1 scores around 0,62–0,64, because entry language is more varied and ambiguous ("let's try it," "here we go," "small size long here"). The rarest class, MOVE_TO_BREAKEVEN, improves to F1 = 0,444 after cleanup but still has only 13 support examples, too few for reliable learning.

The SVM shows the same overall pattern but trades off differently on the entry classes. It is right more often when it does call an entry (precision 0,684 on ENTER_LONG, versus 0,574 for logistic regression), but it calls far fewer of them (recall 0,587 versus 0,717). In plain terms, the SVM only announces an entry when the wording is unmistakable, which makes it more cautious than logistic regression on exactly the classes where wording varies most.

4.4 Transformer Model Performance

Table 9 presents the per-label results for the two transformer models side by side.

Table 9: Per-label precision (P), recall (R), and F1 for the two frozen transformer models on the cleaned dataset. Supp. = support, the number of test examples whose true label is the class in question.

	DistilBERT			ModernBERT			
Label	**P**	**R**	**F1**	**P**	**R**	**F1**	**Supp.**
NO_ACTION	0,825	0,637	0,719	0,855	0,712	0,777	548
ENTER_LONG	0,441	0,565	0,495	0,516	0,533	0,524	92
ENTER_SHORT	0,436	0,542	0,483	0,497	0,611	0,548	144
TRIM	0,651	0,756	0,700	0,714	0,826	0,766	242
EXIT_ALL	0,720	0,738	0,729	0,709	0,771	0,739	244
MOVE_STOP	0,681	0,862	0,761	0,821	0,830	0,825	94
MOVE_TO_BREAKEVEN	0,429	0,231	0,300	0,667	0,462	0,546	13

Both frozen transformer feature extractors show lower precision on entry classes than the two linear models. DistilBERT achieves 0,441 precision on ENTER_LONG (versus 0,574 for logistic regression), while ModernBERT reaches 0,516. The transformers do achieve relatively high recall on several action classes; DistilBERT detects 86,2% of MOVE_STOP actions, but the frequency of false positives lowers their macro F1.

ModernBERT achieves the highest F1 on MOVE_TO_BREAKEVEN (0,546) of all five models, despite the extreme rarity of this class. This suggests that its contextual embeddings capture some similarity between "breakeven" language and other stop-management language, although this advantage on one rare label does not translate into the best aggregate score.

Across all five models, the per-label F1 scores follow the same difficulty hierarchy: management labels are classified reliably, entry labels are consistently harder and vary more between models, and MOVE_TO_BREAKEVEN remains unstable with only 13 support examples.

4.5 Confusion Matrix Analysis

A confusion matrix is the most direct way to see what kinds of mistakes a model makes. Each row collects the examples whose true label is the row's class, and each column shows which label the model predicted. Correct predictions therefore lie on the diagonal, and every off-diagonal cell counts one specific kind of error. Figure 3 shows the confusion matrices for logistic regression, the best model by macro F1 and action F1, and for ModernBERT, the stronger of the two transformers, both aggregated across all five folds on the cleaned dataset.

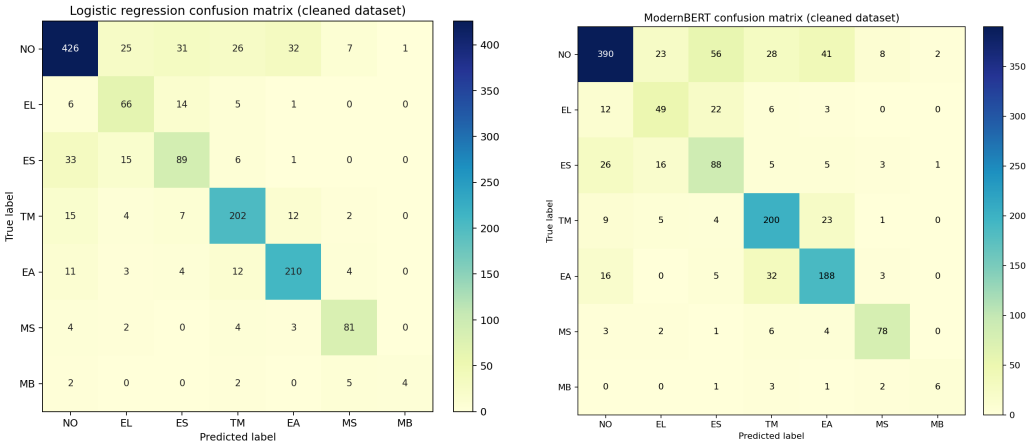

Figure 3: Confusion matrices for logistic regression (left) and ModernBERT (right) on the cleaned dataset, aggregated across all five folds. Rows are true labels, columns are predicted labels, so the diagonal counts correct predictions; darker cells contain more examples. NO = `NO_ACTION`, EL = `ENTER_LONG`, ES = `ENTER_SHORT`, TM = `TRIM`, EA = `EXIT_ALL`, MS = `MOVE_STOP`, MB = `MOVE_TO_BREAKEVEN`.

Several error patterns stand out in the logistic regression matrix.

Confusing the direction of an entry. 14 of 92 true `ENTER_LONG` examples are misclassified as `ENTER_SHORT`; conversely, 15 of 144 true `ENTER_SHORT` examples are misclassified as `ENTER_LONG`. Entry language is often structurally similar regardless of direction, differing only in a directional keyword ("long" versus "short") that may be transcribed ambiguously or may be missing from the relevant text window.

Missed actions. Between 2 and 33 examples of each action class are misclassified as `NO_ACTION`. This happens when the transcript contains an action but the language is too subtle for the classifier, for example a trader who says "I'm going to take a little off" rather than using the explicit keyword "partial."

False alarms. 122 of 548 `NO_ACTION` examples (22,3%) are assigned an action label. Some non-action segments contain language that superficially resembles action language, for example when the trader discusses a hypothetical trade, reviews a past action, or gives advice to viewers. Because the corpus has incomplete action coverage (Section 3.4), a small number of these apparent false positives may also be genuinely unlabelled actions. The rare `MOVE_TO_BREAKEVEN` class barely registers in these totals; with 13 examples it stays unstable, and its most frequent error, landing on `MOVE_STOP`, is at least the most closely related class.

The ModernBERT matrix (right panel) shows the opposite tendency. Logistic regression classifies 426 of the 548 `NO_ACTION` examples correctly; ModernBERT manages only 390 and DistilBERT only 349, because both more often place a non-action line into one of the action columns. In other words, when the frozen transformers are unsure, they lean towards predicting an action. That is exactly why their action recall is high and their precision low in Table 6: they catch more of the real actions, but they also raise more false alarms. The two remaining models lean the other way: the SVM behaves like a stricter logistic regression, and the balanced MLP is the most conservative of all, sending 27 true `ENTER_LONG` and 50 true `ENTER_SHORT` examples to `NO_ACTION`.

4.6 Feature Importance Analysis

To understand *why* TF-IDF-based models do so well, the learned logistic regression weights were inspected directly. The ranking considers every feature the model uses, single words and two-word phrases alike. Figure 4 shows the five highest-weighted features for three representative action classes.

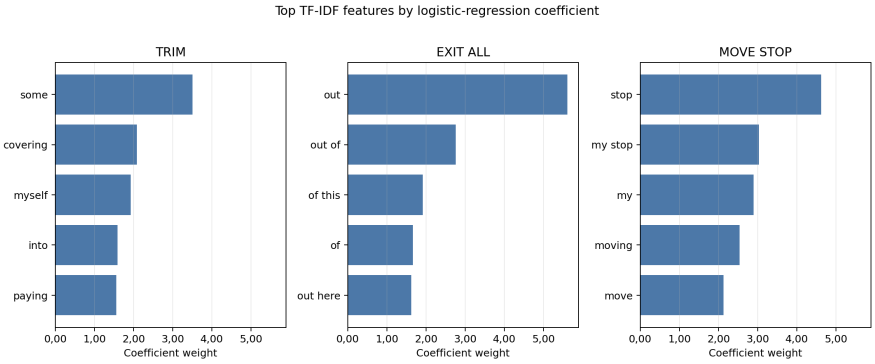

Figure 4: Top-5 TF-IDF features by logistic regression coefficient for three action classes. Each subplot shows the five most predictive words or bigrams for one class. A single token ("out") dominates `EXIT_ALL` with weight +5,61, while "stop" dominates `MOVE_STOP` at +4,62. All subplots share the same x-axis scale, making the relative dominance of top features directly comparable across classes.

The feature weights reveal that trading action language is built from a small set of fixed, recurring phrases. `EXIT_ALL` is dominated by the single word "out" (weight +5,61); `MOVE_STOP` by "stop" (+4,62); `TRIM` by "some" (+3,51). These sparse, high-signal features are captured directly by TF-IDF, whereas in a frozen transformer embedding the same keywords are blended with

everything else in the segment into one dense 768-number summary, where their signal is diluted. The position-state bigrams ("position flat," "position short") also carry high weight, confirming that the structured prompt design (Section 3.2) contributes real classification signal.

4.7 Statistical Significance

Mean scores alone do not show whether a difference between two models is real or just luck with the particular folds. A standard check is the paired t-test: for two models scored on the same five folds, it asks whether the per-fold differences point consistently in one direction or could plausibly be chance. Its output, the p-value, is the probability of seeing differences at least this large if the models were actually equally good; by convention, values below 0,05 count as evidence of a real difference. With only five folds these tests have little power, meaning only large and consistent differences can reach significance, so they are reported as a sanity check rather than as proof.

Each pair of models was compared with such a test on its per-fold macro F1 scores, using the same model versions as Table 6 (for the MLP, the balanced one). These pairwise comparisons ask the natural question of this benchmark: is model A really better than model B, or only on average? The answer is sobering. The two linear models are statistically indistinguishable ($p = 0{,}54$), and neither separates significantly from the balanced MLP. Of the classical-versus-transformer comparisons, only logistic regression versus DistilBERT reaches conventional significance ($p = 0{,}014$); the comparisons with ModernBERT do not. (One technical note: when many tests are run at once, a few can look significant purely by chance, a problem known as multiple comparisons. No correction for it is applied here, since with six tests, of which five are negative anyway, it would not change the picture.)

Looking at the individual folds shows why the tests stay cautious: the linear models win on average, but ModernBERT takes the best macro F1 in one fold, and the transformers take the best action recall in several. The fair summary is that the linear models are ahead overall, and the feature-importance and embedding analyses explain why, but the gap is not so large that it shows up consistently in every single fold.

4.8 MLP Performance with Balanced Class Weighting

The unbalanced MLP fails entirely on `MOVE_TO_BREAKEVEN` (F1 = 0,000) and depresses minority-class recall; balanced oversampling (Section 3.7) lifts macro F1 to 0,634 and `MOVE_TO_BREAKEVEN` F1 to 0,133, with the gains concentrated in the entry classes (`ENTER_LONG` F1 0,533, `ENTER_SHORT` 0,590)

while the management classes stay strong (`MOVE_STOP` 0,852, `TRIM` 0,799, `EXIT_ALL` 0,798). Even so, the MLP still trails both linear models (macro F1 0,634 versus 0,703 and 0,688). The remaining gap is consistent with inefficient use of capacity: with 5,4 million trainable parameters but only about 1.150 training examples per fold, the MLP's extra flexibility does not translate into better generalization. This matches the broader observation that nonlinear models usually need substantially more data to beat well-regularized linear classifiers on high-dimensional sparse features [14].

4.9 Embedding Visualization

If the frozen encoders organized trading language well, examples with the same label should end up with similar embedding vectors. This can be inspected visually with t-SNE [32], a technique that squashes high-dimensional vectors down to two dimensions for plotting while trying to keep similar vectors close together. If the encoder separated the classes well, points of the same colour would form visible groups in such a plot.

For each of the two transformer models, all 1.377 cleaned, deduplicated examples were embedded with exactly the same procedure as in the classification experiments, so the plots show the very same vectors that the classification heads had to work with, projected down to two dimensions with t-SNE. Figure 5 shows the result.

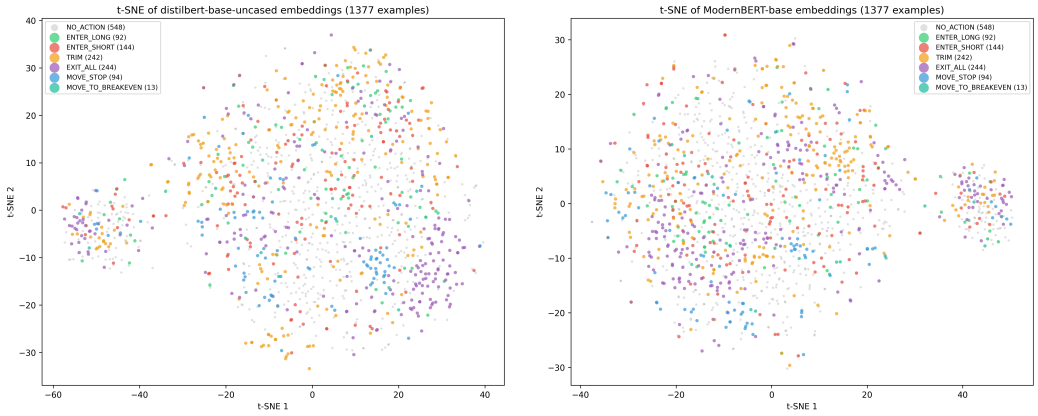

Figure 5: t-SNE projections of frozen DistilBERT (left) and ModernBERT (right) embeddings for all 1.377 cleaned examples, coloured by true label.

Both plots show no clear label-level clustering. All seven classes are heavily intermixed: `NO_ACTION` spreads across the whole projected space, and the action classes overlap strongly with one another and with `NO_ACTION`.

DistilBERT retains slightly more loose sub-structure, but those groupings contain mixed labels and likely reflect transcript-level or syntactic similarity rather than trading intent, while ModernBERT's cloud is even more uniform. In both, `MOVE_STOP` is the most spatially coherent class, consistent with the near-ubiquity of the word "stop" in its examples. t-SNE is a visualization, not a formal test, so this evidence is illustrative; but it fits the picture from the feature-importance analysis. "Peeling some off" (`TRIM`) and "watching for longs" (`NO_ACTION`) can be close neighbours in general English, yet they need different labels; TF-IDF keeps them apart as distinct sparse features, whereas the frozen embeddings may place them side by side.

4.10 Error Analysis

To characterize the practical failure modes of the best-performing model, all misclassifications from logistic regression's five-fold cross-validation on the cleaned dataset were collected: 299 errors out of 1.377 examples (21,7%). Importantly, every one of these errors comes from the test side of a fold: each example was classified by a model that had never seen its transcript during training, so these are genuine generalization errors, not mistakes measured on training data. One caveat from the label check (Section 3.4) carries over: the labelled actions are reliable, but not every real action carries a label. A prediction counted as a false positive against `NO_ACTION` can therefore occasionally be a hit on one of those unlabelled actions rather than a genuine mistake. Table 10 lists the ten most frequent error patterns.

Table 10: Top 10 misclassification patterns for logistic regression on the cleaned dataset.

True Label	Predicted	Count	% of Errors
ENTER_SHORT	NO_ACTION	33	11,0
NO_ACTION	EXIT_ALL	32	10,7
NO_ACTION	ENTER_SHORT	31	10,4
NO_ACTION	TRIM	26	8,7
NO_ACTION	ENTER_LONG	25	8,4
ENTER_SHORT	ENTER_LONG	15	5,0
TRIM	NO_ACTION	15	5,0
ENTER_LONG	ENTER_SHORT	14	4,7
EXIT_ALL	TRIM	12	4,0
TRIM	EXIT_ALL	12	4,0

Examination of the misclassified examples reveals three dominant failure modes.

Indirect entry language. The most common error type (33 cases of `ENTER_SHORT` → `NO_ACTION`) arises when traders describe entries using past-tense, implicit, or unusual phrasing. For example, "I did enter short versus New York here" uses past tense; "I've re-entered versus flow zones now" uses an uncommon verb form; and "All right. In it scalp here" lacks an explicit directional keyword. The classifier has learned to trigger on standard patterns like "short here" or "small short," and misses these variations.

Exit-adjacent commentary. The second pattern (32 cases of `NO_ACTION` → `EXIT_ALL`) occurs when the model over-triggers on exit-related language in non-action segments. Typical examples include "Stopped me out by a point or two" (describing a past stop-out, not a live action), "Any push through 11s here. I'm out" (a conditional statement), and "There we build out this right shoulder" (technical analysis where "out" appears in a non-exit context). The strong weight of the keyword "out" (+5,61) makes the model sensitive to any occurrence, including non-action uses.

Advisory framing. The third pattern (26 cases of `NO_ACTION` → `TRIM`) involves second-person advice to viewers that superficially resembles self-directed action: "Must be taking partials here, folks," "Cover a little bit here into 600 just in case," and "You can cover a piece here." The keywords "partials," "cover," and "piece" are strong `TRIM` signals, but the second-person framing ("you can," "folks") indicates advice rather than a trade the speaker is executing. The current text normalization pipeline does not distinguish between first-person and second-person constructions.

The error patterns point to concrete improvements: the model needs to pay more attention to verb tense (did something already happen, or is it happening now?), to who is being addressed (is the trader acting, or telling viewers what to do?), and to the difference between a plan and an executed action.

4.11 How Large Does the Vocabulary Need to Be?

Here, vocabulary means the list of distinct words and word pairs that the TF-IDF step turns into features; its size was capped at 20.000 in all experiments. A final check verifies that this cap is neither a hidden bottleneck nor an inflated setting. The method is an *ablation*: the same experiment is re-run several times with only one setting changed, here the maximum number of features. Logistic regression was evaluated with five-fold cross-validation on the cleaned dataset at caps of 5.000, 10.000, 20.000, and 50.000 features, yielding macro F1 of 0,700, 0,703, 0,703, and 0,703 respectively (accuracy

0,781/0,785/0,785/0,785; action F1 0,885/0,887/0,887/0,887). Performance saturates at 10.000 features, and raising the cap beyond that changes nothing, simply because the corpus only contains about 10.000 distinct unigrams and bigrams in the first place. Even 5.000 features capture 99,7% of the top performance. The classification signal is concentrated in a compact set of domain-specific terms, and the 20.000-feature default is more than sufficient.

4.12 From the Benchmark to Complete Sessions

All results so far come from the AI-labelled dataset, in which trade actions are heavily overrepresented (Section 3.4). The final experiments therefore ask the question that actually matters for deployment: what does the best model do when it has to read a real session from start to finish? Five complete livestream sessions were replayed line by line, 18.709 transcript lines covering 15,1 hours of streaming, and none of these lines were used for training anywhere in this thesis. Logistic regression was trained once on the cleaned dataset minus these five sessions and then asked to classify every single line, with prompts built by the same machinery that produced the training data.

The replay answers two questions. First, does the model still find the actions it should find? Largely yes: of the 23 labelled actions in these sessions, 18 (78%) received an action prediction, 15 of them with exactly the right label, broadly in line with the action recall measured on the benchmark.

Second, and decisively: what else does it fire on? Here the picture collapses. Across the unlabelled lines, the model predicted an action 3.395 times, about 225 times per hour, while a day trader performs perhaps five to ten real actions per hour. The benchmark could not reveal this, because of a base-rate problem: when the thing being searched for is rare, even a decent detector produces mostly false alarms, just as a reliable medical screening test still produces mostly false positives when the disease itself is rare. In the benchmark, roughly every third line was an action; in a real session, fewer than one line in a hundred contains one.

How many of the 3.395 fires were actually right? To estimate this, a random sample of 100 fires (20 per session) was reviewed against the surrounding transcript. The review worked the same way the dataset itself was built: an AI model (Claude) read several lines of context around every fire and judged it, and the author then checked and confirmed the verdicts; every verdict is recorded with its reason in the evaluation files. The result: 3 of the 100 sampled fires were real trade actions that the dataset had simply never labelled, 5 were borderline, and 92 were clear false alarms. The model's standalone precision on a full stream is therefore roughly 3%, and at most 8% if every borderline case is counted in its favour, against 0,863 on the

benchmark. The false alarms read like the error analysis of Section 4.10 at scale: market analysis that contains trade words, reviews of past trades, conditional plans, and advice to viewers.

Could a stricter decision threshold fix this? No. Only 3 of the 3.395 false fires exceed the production system's confidence gate of 0,74, which sounds promising until one checks the other side: only 1 of the 18 correctly detected actions exceeds that gate as well. The model is unsure about everything, false alarms and real actions alike, so any threshold strict enough to silence the noise also silences the signal. Run on its own, the classifier is unusable on a real stream. How the production system avoids this fate is measured next.

Two design notes are needed for honesty. The position state during the replay was rebuilt from the 23 labelled actions, and since those are sparse, the state says "no open position" most of the time; this mirrors how the training prompts were built, but it pushes the model's mistakes towards entry predictions, which expect no open position. And the replay itself was checked for fidelity by comparing the prompts it rebuilds against the stored training prompts of these sessions: most match character for character, so the model really saw its familiar input format.

4.13 The Full Pipeline on the Same Sessions

The numbers above describe the classifier running alone, which is not how the production system uses it. In production, the primary decision is made by the rule engine, the layer of several hundred hand-written text patterns introduced in Section 3.1, and the classifier only assists: its confidence scores are used to veto doubtful rule matches and to recover actions the rules missed. As a closing comparison, the same five sessions were replayed through this real interpretation pipeline in three configurations: the rule engine alone, the rule engine with the deployed ModernBERT classifier, and the rule engine with the benchmark winner, TF-IDF logistic regression, in the classifier slot. During the replay, the position state evolved from the pipeline's own decisions, as it would in live trading, and a fired intent counts as detecting a labelled action when it lands within two lines of the labelled line. Table 11 shows the outcome next to the standalone result.

Table 11: The production interpretation pipeline on the five held-out sessions (15,1 hours), compared with the standalone classifier of Section 4.12. Fires are action intents on lines that carry no action label.

Configuration	Labelled actions detected	Exact label	Fires per hour
Standalone classifier (Section 4.12)	18 / 23	15	∼225
Rule engine alone	5 / 23	5	5,4
Rule engine + ModernBERT (production)	7 / 23	7	6,2
Rule engine + TF-IDF logistic regr.	7 / 23	7	6,3

Three things stand out.

First, the pipeline operates in an entirely different regime: roughly six fires per hour instead of 225, a thirty-five-fold reduction. And these fires are no longer mostly noise. Every fire of all three configurations (99 distinct transcript lines) was reviewed against the surrounding transcript, with the same AI-plus-author review process as the standalone sample. Of the 93 fires of the production configuration, 15 turned out to be real trade actions that simply carry no label in the dataset (live announcements such as "I'm out of this now" or "stops into break even"), 8 were borderline, mostly the trader reporting an action he had executed moments earlier, and 70 were false alarms. So about one fire in six hits a real action, against roughly one in thirty for the standalone model, and the false alarms drop from around 210 per hour to about 4,6 per hour.

Second, the caution has a price. The pipeline detects only 7 of the 23 labelled actions, far below the standalone model's 18, because the rule engine refuses exactly the subtle phrasings ("Okay. Out. Oh, let it squeeze out.") that the statistical model catches. What it does catch, it labels exactly right, every time. The system is built to prefer missing a trade over inventing one, and the numbers show that choice plainly.

Third, the classifier earns its supporting role, but which model fills that role makes almost no difference. Adding either classifier lifts detection from 5 to 7 of 23, at the cost of about one additional fire per hour, and the deployed ModernBERT and the benchmark-winning TF-IDF model produce nearly identical system behaviour, down to the share of real actions among their fires. Once the rules make the primary decision, the performance gap between the model families almost disappears.

What does this mean for practical use? As a fully automatic trade copier, the pipeline is not good enough yet: it would still trigger around four to five unjustified trades per hour while missing two thirds of the labelled actions. As an alerting assistant, where a human sees each proposed trade and confirms

or rejects it, the numbers are workable: the alert rate is low enough to follow live, about every sixth alert points at a genuine action the dataset never knew about, and the detections it does make are labelled exactly right. The honest verdict is that the layered design turns an unusable classifier into a usable assistant, but not into an autonomous system.

5 Discussion

5.1 Why the Classical Models Win Here

The most prominent finding of this work is that the TF-IDF-based linear models outperform the frozen transformer-head models on the aggregate metrics. Given how thoroughly transformers dominate recent NLP benchmarks, this deserves a short explanation, and the evidence for it has already appeared along the way.

Three strands of the explanation reinforce each other. The TF-IDF features are learned entirely from the training data, so they capture the domain's vocabulary automatically, while the frozen encoders, trained on general-purpose text, never had a reason to tell trading phrases such as "peeling some off" (a `TRIM`) and "taking this off" (`TRIM` or `EXIT_ALL`, depending on context) apart. Trading language is also unusually repetitive: a small repertoire of fixed phrases announces most actions, and bigram features give each of these phrases its own dedicated dimension, which is exactly the sparse keyword signal the feature-importance analysis surfaced. And the comparison is asymmetric by design: the classical models adapt every parameter to the task, while the transformers may only train their tiny heads, so the part of them that can actually specialize in trading language is roughly ten times smaller than a linear TF-IDF model. The t-SNE plots close the loop: the frozen embeddings leave the seven classes intermixed, and a thin trained head cannot fix a space that never separated the classes in the first place. The result is consistent with the broader literature, where simple baselines remain competitive with deep learning on small or domain-specific text classification tasks [14].

An important caveat applies. This study evaluates transformers exclusively in the feature-extraction setting (frozen encoder, trained head). The results show that *frozen pretrained embeddings* do not beat TF-IDF on this task; they do not show that transformers in general are inferior. A fully fine-tuned transformer, or one adapted with parameter-efficient methods such as LoRA, might close or reverse the gap. The comparison here is between fully task-adapted classical models and minimally task-adapted transformers, which

is not a level playing field for the transformers; it is, however, the realistic comparison when the dataset is too small to fine-tune safely.

5.2 The Precision-Recall Tradeoff

The five models spread out along a clear precision-recall spectrum for action detection. The SVM and the balanced MLP favour precision (0,900 and 0,898 action precision), while ModernBERT favours recall (0,917 action recall). Logistic regression occupies the most balanced position, with action precision 0,863 and action recall 0,914, giving it the strongest action F1.

This tradeoff has direct practical implications, because the two error types cost different things. A false positive means executing a trade the trader never made; a false negative means missing a trade he did make. A deployment that prioritizes safety would favour a precision-leaning model such as the SVM, accepting that some signals are missed. A deployment that prioritizes completeness would favour high recall (the transformers), accepting more false alarms, which then must be caught by downstream safeguards. The balanced profile of logistic regression makes it the most versatile default.

5.3 Which Actions Are Hard, and Why

The per-label results tell one consistent story across all five models. Management actions (`MOVE_STOP`, `EXIT_ALL`, `TRIM`) are the easiest, with F1 around 0,70–0,85, because traders announce them with explicit, almost ritual words: "stop," "out," "partial." The `NO_ACTION` class sits in the middle (F1 roughly 0,72–0,83), and varies across models because the boundary between no action and a subtly phrased action is genuinely blurry, all the more so given the incomplete action coverage of the labels. Entries (`ENTER_LONG`, `ENTER_SHORT`) are consistently hard (F1 roughly 0,48–0,64): entry language is varied and often implicit, and the model must also catch the direction, which may hide in an earlier sentence. `MOVE_TO_BREAKEVEN`, finally, remains the unstable outlier for the reason established earlier: there is simply almost no data for it.

This hierarchy suggests where improvement is cheapest. Entry classification would benefit most from additional training examples with diverse phrasing. For `MOVE_TO_BREAKEVEN`, the most pragmatic fix is structural rather than statistical: it could be merged into the closely related `MOVE_STOP` class, since moving a stop to breakeven is a special case of moving a stop.

5.4 Implications for Production Deployment

The benchmark winner is logistic regression, yet the production system runs ModernBERT in the classifier slot, a choice made before this benchmark existed. Should production simply switch to the best model? Looked at in isolation, yes: logistic regression classifies somewhat more accurately at no extra cost. But the full-stream experiments showed that the question matters less than it seems. No standalone classifier of either family could carry the live system, because on realistic input the false alarms vastly outnumber the real signals and no confidence threshold separates them. The architecture that follows is not a preference but a necessity: the rule engine, which fires rarely and precisely, makes the primary decision, and the classifier seconds it, vetoing doubtful rule matches and recovering clear misses. In that supporting role, the pipeline replay showed both models producing nearly identical system behaviour, so swapping the deployed model would change little. The deciding factors become practical ones instead, such as memory use and how easy the trained model is to ship, and an end-to-end evaluation including live execution would still be needed before swapping any component. The real lever for improving the system is not the choice between these two classifiers but the training data itself (Section 5.7).

Latency deserves its own mention, because in trading it matters nearly as much as accuracy: a signal that arrives seconds late gets executed at a worse price, and the production risk engine deliberately discards signals that are too old. Within the pipeline, by far the slowest part is getting from speech to final text. The system first has to wait for the trader to finish the utterance (segments run up to about four seconds), transcription itself adds a few hundred milliseconds, and the transcription queue is kept deliberately short so that a fast talker cannot build up a backlog of stale audio. Classification, in contrast, takes only milliseconds to a few tens of milliseconds, for the classical models and the transformer head alike. Model choice therefore barely moves the end-to-end latency; speech-to-text dominates it, and any future latency work should start there.

5.5 Ethical, Legal, and Operational Risk

The application context raises risks that are separate from classification accuracy. A false positive in a trade-copying system is not merely a labelling error; if connected to execution infrastructure, it could create an unintended market order. Any production deployment should therefore keep human override controls, confidence thresholds, stale-signal checks, position-size limits, and audit logs in place. The evaluation in this thesis measures

text classification only and should not be interpreted as evidence that fully automated execution is safe.

The data source also requires care. Livestream transcripts are public-facing media, but automated collection and reuse may still be constrained by platform terms, copyright, or streamer-specific expectations. This thesis uses archived transcripts for research; an operational system should verify that its collection, storage, and use of transcript data are permitted. Finally, detected trade signals should not be presented as financial advice. The system classifies what a speaker appears to say; it does not evaluate whether the trade is suitable, lawful, or economically sound.

5.6 Limitations

Several limitations of this work should be acknowledged.

Benchmark composition, and the scope of the full-stream test. The primary results (Sections 4.1 to 4.11) were measured on the curated benchmark corpus, in which trade actions are heavily overrepresented, and must not be read as deployment accuracy. The full-stream evaluation (Section 4.12) was added precisely to measure that gap, and it found a severe one: standalone precision of only a few percent on complete sessions. That test has limits of its own, however. It covers five sessions with only 23 labelled actions, so its recall estimate is coarse; the position state during the replay was reconstructed from the sparse labels and was therefore flat most of the time; and the precision estimate rests on a 100-fire sample, reviewed by an AI model and confirmed by the author, rather than on a fully labelled stream. The pipeline replay (Section 4.13) shares these limits, although its fires were at least all individually reviewed with the same AI-plus-author process. The benchmark results remain what they were designed to be, a controlled comparison between model families; the full-stream results give the direction and rough size of the deployment gap, not a precise production error rate.

Dataset size. At 1.442 cleaned examples (1.377 after deduplication) across 103 transcripts, the dataset is small. The results may not generalize to larger or differently distributed datasets, and the relative ranking of classical versus transformer models could change with substantially more training data, particularly if full fine-tuning becomes feasible.

Single domain. All transcripts come from a single trader and a single instrument (Nasdaq futures). Different traders, instruments, or trading styles may use different language that requires separate training.

Frozen encoders. The transformer models were evaluated exclusively with frozen encoder weights. Full fine-tuning, parameter-efficient fine-tuning

(such as LoRA), or continued pretraining on trading-domain text might substantially improve their performance but were not explored.

Imperfect labels. The labels were generated by LLMs and reviewed by the author, and the manual verification (Section 3.4) found them precise but incomplete: not every action in the streams is labelled, and some `NO_ACTION` lines may contain weak or repeated actions. The labels also record only what the trader announced; without broker statements there is no proof that an announced trade was actually placed. Finally, the labelled lines were not sampled at random. Lines with clear trade wording were more likely to be selected for labelling in the first place, and two accepted labels were ambiguous even to a human reader. This selection makes the classes look cleaner than the underlying speech is, and it can make the measured error rates, especially the false-positive rate, look better than they would on unselected speech.

Each line judged in isolation. The classifier treats every transcript segment independently, although trading actions occur in sequences (entry, then management, then exit). A model aware of this temporal structure could rule out impossible predictions and resolve ambiguous lines from their context.

5.7 Future Work

Several directions follow from the findings and limitations.

Training with realistic class balance. The training data presents the model with one action for every 2,5 non-actions, while a real stream contains over a hundred non-action lines per action. The full-stream test (Section 4.12) showed where that mismatch leads. Retraining with far more hard negative examples, precisely the analysis-with-trade-words chatter that causes the false alarms, is the most direct way to close the measured precision gap.

Fully labelled full-stream evaluation. The full-stream test relied on the 23 labelled actions of five sessions and a sampled review of the model's fires. Labelling several entire sessions exhaustively, every line, would turn the sampled precision estimate into an exact one, allow recall to be measured beyond the existing labels, and give the position-state replay a complete action sequence to work with.

Full fine-tuning with more data. Expanding the labelled set to several thousand examples across multiple traders and then fine-tuning the entire encoder, instead of keeping it frozen, is the most direct path to stronger transformer performance; at 5.000–10.000 examples, overfitting becomes manageable with standard regularization.

Parameter-efficient fine-tuning. Methods such as LoRA adapt a pretrained model by training only a small number of additional parameters,

a middle ground between frozen feature extraction and full fine-tuning that could be feasible even at the current dataset size.

Sequence-aware modelling. The classifier currently judges every segment on its own, even though trades unfold in order: an exit can only follow an entry, and a stop can only be moved while a position is open. A model that reads consecutive segments together, or that carries the running position state from one prediction into the next, could rule out impossible actions and recover entries that only become clear over two or three lines.

Domain-specific pretraining. Continued pretraining on a large corpus of unlabelled trading transcripts, which are freely available on YouTube, could adapt the embeddings to trading vocabulary and reduce the mismatch with general-purpose pretraining data.

End-to-end evaluation with execution. Section 4.13 replayed the interpretation pipeline offline. An end-to-end test that adds the optional LLM confirmation layer, live transcription noise, and the risk and execution stages, ideally judged against broker records, remains open.

6 Summary

This work addressed the question of which machine learning approach provides the most accurate trade-intent classification from noisy livestream transcripts, comparing classical TF-IDF-based methods with frozen transformer feature extractors.

Question. Given a short transcript segment from a daytrading livestream and the current trading context, which model family best predicts the trader's intended action from a set of seven classes (no action, enter long, enter short, trim, exit all, move stop, move to breakeven)? Which action classes are most and least amenable to automated detection, and what systematic error patterns emerge?

Procedure. From 103 transcribed YouTube livestreams, 1.466 lines were labelled with the help of three large language models and reviewed by hand; cleanup and deduplication leave 1.377 unique examples for the primary evaluation. A manual check of 203 action labels confirmed nearly all of them, but also showed that not every action in the streams is labelled, so the benchmark is action-enriched rather than a complete record. Five models were compared under identical conditions: logistic regression, a linear SVM, and an MLP on TF-IDF features, and DistilBERT and ModernBERT as frozen encoders with small trained classification heads. Evaluation used five-fold cross-validation, split by transcript so that no model is ever tested on a session it trained on. A closing experiment replayed five complete held-out

sessions, more than 18.000 lines, through the best model.

Results. On the cleaned dataset, logistic regression achieved the best macro F1 (0,703) and action F1 (0,887), while the SVM achieved the best accuracy (0,795) and action precision (0,900). ModernBERT achieved the highest action recall (0,917) but lower precision and macro F1. The MLP improved markedly with balanced oversampling (macro F1 from 0,572 to 0,634) but still trailed both linear models. Data cleanup improved most metrics. The supporting analyses explain the gap: classification is driven by a small set of domain keywords that TF-IDF captures directly, the frozen embeddings show no label-level structure under t-SNE, and the dominant error modes are indirectly phrased entries, exit-like commentary, and advice to viewers mistaken for action. A vocabulary ablation showed performance saturating at about 10.000 features. The full-stream experiment then showed the second central finding of this work: the benchmark numbers do not transfer to complete sessions. Reading five full held-out sessions, the best model still found 18 of 23 labelled actions, but fired about 225 times per hour, of which only a few percent were real actions, so on its own it would be unusable live. The cause is the rarity of real actions in natural speech, and the consequence is the layered production design in which hand-written rules decide and the classifier merely assists. A replay of that full pipeline over the same sessions confirmed the design: it fires only about six times per hour, a review of every one of those fires showed that about one in six points at a real trade action the dataset never labelled, the classifier layer adds detections the rules miss, and the choice of assisting model, ModernBERT or TF-IDF, barely changes the system's behaviour (Sections 4.12 to 5.4).

These results show that for small, domain-specific text classification tasks with repetitive, keyword-driven language, classical machine learning methods on TF-IDF features remain highly competitive. The added complexity of pretrained transformer feature extraction does not translate into better performance when the encoder is frozen and the training set is small.

For practitioners facing similar tasks, classification on small, specialised, and often noisy corpora, the practical lesson is to start with simple, well-understood baselines before investing in heavier architectures, and to evaluate them under conditions as close as possible to deployment. The dominance of pretrained transformers on standard benchmarks does not automatically transfer to every setting; when the encoder cannot be fine-tuned and the classification signal concentrates in a small set of domain keywords, a TF-IDF model with a balanced linear classifier is a competitive, interpretable, and easy-to-deploy choice.

The source code of the system and of the complete benchmark pipeline is publicly available at `https://github.com/AndiCodesss/bachelor_thesis`.

Apart from five example transcripts that show the data format, the recorded transcripts and the labelled dataset are not included in the repository, mainly because of their size, so the repository documents the pipeline and its stored results rather than providing a rerunnable copy of the experiments (Section 3.8.3).

References

[1] Johan Bollen, Huina Mao, and Xiaojun Zeng. Twitter mood predicts the stock market. *Journal of Computational Science*, 2(1):1–8, 2011.

[2] Cristian Bucilă, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 535–541, 2006.

[3] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[4] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[5] Tri Dao, Dan Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 35, 2022.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[9] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. *Applied Logistic Regression*. John Wiley & Sons, 3rd edition, 2013.

[10] Minqing Hu and Bing Liu. Mining and summarizing customer reviews. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 168–177. ACM, 2004.

[11] Thorsten Joachims. Text categorization with support vector machines: Learning with many relevant features. In *European Conference on Machine Learning*, pages 137–142. Springer, 1998.

[12] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[13] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, volume 2, pages 1137–1143, 1995.

[14] Kamran Kowsari, Kiana Jafari Meimandi, Mojtaba Heidarysafa, Sanjana Mendu, Laura Barnes, and Donald Brown. Text classification algorithms: A survey. *Information*, 10(4):150, 2019.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.

[16] Tim Loughran and Bill McDonald. When is a liability not a liability? textual analysis, dictionaries, and 10-Ks. *The Journal of Finance*, 66(1): 35–65, 2011.

[17] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[18] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[20] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[21] Jeffrey Pennington, Richard Socher, and Christopher D Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1532–1543, 2014.

[22] Bethany Percha. Modern clinical text mining: A guide and review. *Annual Review of Biomedical Data Science*, 4:165–187, 2021.

[23] Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, 2018.

[24] Matthew E Peters, Sebastian Ruder, and Noah A Smith. To tune or not to tune? adapting pretrained representations to diverse tasks. In *Proceedings of the 4th Workshop on Representation Learning for NLP (RepL4NLP-2019)*, pages 7–14, 2019.

[25] John C Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J Smola, Peter L Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 2000.

[26] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *PMLR*, pages 28492–28518, 2023.

[27] Alexander Ratner, Stephen H Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017.

[28] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5): 513–523, 1988.

[29] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[30] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009.

[31] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[32] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[34] Mayur Wankhade, Annavarapu Chandra Sekhara Rao, and Chaitanya Kulkarni. A survey on sentiment analysis methods, applications, and challenges. *Artificial Intelligence Review*, 55(7):5731–5780, 2022.

[35] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[36] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[37] Frank Z Xing, Erik Cambria, and Roy E Welsch. Natural language based financial forecasting: A survey. *Artificial Intelligence Review*, 50 (1):49–73, 2018.

[38] Yi Yang, Mark Christopher Siy Uy, and Allen Huang. FinBERT: A pretrained language model for financial communications. *arXiv preprint arXiv:2006.08097*, 2020.